DATA SCIENCE TECHNOLOGY

Homework:
Polyglot Persistence for a Next-Gen MMORPG

Enrico Madonna
Matr. 0120000380

Professor: A. Maratea
A.Y. 2025/2026

June 25, 2026

Abstract

This homework proposes a data storage architecture for *Loca Lucha*, a next-generation Massively Multiplayer Online Role-Playing Game (MMORPG) operating at global scale. The goal is to design, at an architectural level only, a polyglot persistence solution that can simultaneously support: strongly-consistent financial transactions, low-latency real-time gameplay, durable storage of player profiles and inventories across regions, and large-scale analytical processing of telemetry logs. The proposed design combines four different technologies: PostgreSQL, Redis, Apache Cassandra and a Data Lake queried via Apache Hive. Each assigned to the subset of data and workload it is natively best suited for.

1 Overview

This proposal focuses exclusively on the *data storage* aspects of the backend for *Loca Lucha*, a fictitious MMORPG developed by Goofysoft and deployed worldwide. The aim is to reason about which storage technologies should be used for which data, and to justify these choices in terms of consistency, latency, availability and scalability requirements.

The system is expected to handle tens of millions of registered players, with peaks of a few hundred thousand concurrent users (CCU) distributed across several geographical regions (Europe, North America, Asia). From a data management perspective, the backend must support four main tasks:

- **Account and billing management:** player accounts, authentication, wallets and cash-shop transactions involving real money or premium currency.
- **Real-time gameplay state:** low-latency management of sessions, matchmaking and player-vs-player (PvP) leaderboards.
- **Persistent game state:** durable storage of character profiles, inventories, quest progress and in-world coordinates across multiple regions.
- **Telemetry and analytics:** ingestion and long-term storage of large volumes of gameplay logs for balancing, fraud detection and business intelligence.

These tasks have heterogeneous and partially conflicting requirements. For example, strong consistency is mandatory for financial data, while ultra-low latency is the dominant concern for PvP state updates; persistent character data must remain available even in the presence of network partitions, whereas telemetry logs mainly require very high write throughput and cheap long-term storage.

For this reason, the backend adopts a *polyglot persistence* approach: the data layer is decomposed into four logical stores, each backed by a different technology and assigned to a specific class of data and workload:

- a relational OLTP cluster (**PostgreSQL**) for account and financial data, where ACID properties and strong consistency are non-negotiable;
- an in-memory key–value store (**Redis**) for session state and real-time leaderboards, where sub-millisecond latency is critical;
- a wide-column NoSQL store (**Apache Cassandra**) for globally-distributed player profiles and inventories, where high availability and tunable consistency are required;
- a cloud-based **Data Lake** queried with **Apache Hive** for telemetry and log data, where append-only writes and analytical batch processing over terabytes of historical data are the dominant workload.

2 Domain and Data Classification

This section briefly describes the logical architecture of the Loca Lucha backend and classifies the data that must be stored. The classification follows the four-way split proposed in the literature for MMORPGs: *Account Data*, *Game Data*, *State Data* and *Log Data* [1]. Each class will later be mapped to a different storage technology.

2.1 Application Domain: Loca Lucha MMORPG

Loca Lucha adopts the typical client–server model used by large-scale MMORPGs. Players connect from PC or console clients to a set of gateway servers (load balancers), which then route them to *zone servers* located in the closest data center. Each zone server maintains a portion of the virtual world and is composed of multiple internal components (map servers, logic servers) running the game rules.

At a high level, the main server-side components interacting with the data layer are:

- a **Login service**, responsible for authenticating users and interacting with the account database;
- a **Game service**, running inside each zone server, which loads and updates character state, manages matchmaking and PvP arenas;
- a **Store service**, in charge of cash-shop operations and premium currency transactions;
- a **Telemetry service**, which collects gameplay and system events and forwards them to the analytics pipeline.

These services do not share a single monolithic database; instead, they interact with multiple data stores according to the class of data they need to read or write.

2.2 Data Classes

Table 1 summarises the four data classes considered in this design, together with examples of attributes, access frequency, read/write patterns and expected lifetime.

Table 1: Main data classes in Loca Lucha and their usage patterns.

Data class	**Example attributes**	**Access frequency**	**Read/write pattern**	**Data lifetime**
Account Data	UserID, credentials, email, wallet balance, recharge history, subscription status	Low–medium (login/logout, payments)	Point reads and small ACID transactions	Very long (lifetime of the account, years)
Game Data	Item and NPC metadata, class definitions, map geometry, quest scripts, balancing parameters	Frequent reads, very rare writes (patches)	Mostly read-only, occasional batch updates	Long, with occasional versioned changes
State Data	Character level and stats, inventory contents, quest progress, position, temporary buffs/debuffs	Very high during gameplay	Burst real-time reads/writes, hot state cached in memory, periodic persistence	Medium: current state plus recent history
Log Data	Gameplay events (kills, deaths, joins, leaves), errors, anti-cheat signals, chat events, store events	Extremely high (one or more events per action)	Append-only writes at high throughput, mostly batch reads for analytics	Very long (full history kept for months or years)

This classification is the conceptual bridge between the application domain and the storage layer. In the next section, each class will be quantified in terms of data volume and access pattern (NU, DAU, CCU), providing the basis to motivate the choice of different storage technologies for each subset of data.

3 Data Nature and Volume in the MMORPG Context

In order to justify the choice of different storage technologies, we estimate the main data volumes and access patterns for Loca Lucha. The numbers are approximate but realistic for a successful global MMORPG and are sufficient to highlight which workloads are small but critical, which are write-heavy, and which are truly "big data".

These assumptions are consistent with studies on data management for MMORPGs, which show that traditional RDBMS architectures become a bottleneck for *state* and *log* data as the number of players and the volume of events grow, motivating the adoption of cloud and NoSQL storage systems alongside relational databases [1, 2].

3.1 Global Load Parameters

We assume the following aggregate figures:

- Number of registered users (NU): 10,000,000.
- Daily Active Users (DAU): 1,000,000.
- Peak Concurrent Users (CCU): 200,000.
- Logical disk block size: $B = 4096$ bytes (4 KB), used for rough blocking–factor estimates in the relational part.

These parameters will be used as a reference for the storage estimates in the following subsections.

3.2 Relational Layer: Accounts and Transactions (PostgreSQL)

The relational layer stores the core account information and the cash-shop transaction log.

Account table. Each account record is assumed to contain:

- `user_id` (8 bytes),
- `email` (64 bytes),
- password hash (64 bytes),
- wallet balance (8 bytes),
- registration timestamp (8 bytes).

The average record size is therefore $R_{\text{acc}} \approx 152$ bytes. With a 4 KB page size, the blocking factor is

$$BFR_{\text{acc}} = \left\lfloor \frac{4096}{152} \right\rfloor = 26 \text{ tuples per page.}$$

For 10,000,000 accounts this yields

$$NP_{\text{acc}} = \left\lceil \frac{10{,}000{,}000}{26} \right\rceil \approx 384{,}616 \text{ pages.}$$

Transaction table. For cash-shop operations we assume a record structure:

- `tx_id`, `user_id`, `item_id`, `price`, `timestamp` (8 bytes each).

The average record size is $R_{\text{tx}} \approx 40$ bytes. For an order of 50,000,000 transactions we obtain:

$$BFR_{\text{tx}} = \left\lfloor \frac{4096}{40} \right\rfloor = 102 \text{ tuples per page},$$

$$NP_{\text{tx}} = \left\lceil \frac{50{,}000{,}000}{102} \right\rceil \approx 490{,}197 \text{ pages}.$$

The total volume of the main relational tables is then:

$$DV_{\text{SQL}} \approx (384{,}616 + 490{,}197) \times 4096 \approx 3.58 \text{ GB}.$$

This confirms that the financial layer is relatively small in size, but, as discussed later, *critical* in terms of business impact.

3.3 State Persistence: Profiles and Inventories (Cassandra)

Persistent character state (profiles and inventories) is stored in a wide-column NoSQL database. Each player owns on average three characters, leading to approximately 30,000,000 character records. For each character we store a denormalised row containing inventory, statistics, quest progress and position.

We assume an average row size of $R_{\text{prof}} \approx 1.2$ KB, so the logical data volume per replica is:

$$DV_{\text{NoSQL}} \approx 30{,}000{,}000 \times 1.2 \text{ KB} \approx 36 \text{ GB}.$$

Unlike relational systems, Cassandra uses a Log-Structured Merge Tree (LSM) storage engine: updates go to in-memory memtables and are flushed to immutable SSTables on disk, typically compressed in fixed-size chunks. For high availability we assume a replication factor $RF = 3$; the total physical volume before compression is then:

$$DV_{\text{physical}} \approx 36 \text{ GB} \times 3 \approx 108 \text{ GB}.$$

This is a moderate footprint for a small Cassandra cluster, but much larger than the relational layer and clearly in the "high-volume, write-intensive" category.

3.4 Session and Leaderboards (Redis)

Session and matchmaking data are kept in memory for the CCU players and for the top portion of the PvP leaderboard.

- Session tokens: 200,000 keys, 256 bytes each $\Rightarrow \approx$ 51.2 MB.
- PvP leaderboard: 100,000 entries in a Sorted Set, 64 bytes each $\Rightarrow \approx$ 6.4 MB.

The total in-memory footprint is therefore:

$$DV_{\text{RAM}} \approx 51.2 \text{ MB} + 6.4 \text{ MB} \approx 57.6 \text{ MB}.$$

This volume is trivial for a modern server and justifies the use of Redis as a pure in-memory store tuned for sub-millisecond access times, without worrying about memory exhaustion.

3.5 Telemetry and Logs (Data Lake + Hive)

Finally, gameplay and system telemetry are ingested into a Data Lake and queried with Hive for analytics. We assume that each Daily Active User emits around 500 telemetry events per day (actions, movements, errors, chat, store events):

- Number of events per day: $500 \times 1{,}000{,}000 = 500{,}000{,}000$.
- Average compressed event size: 64 bytes.

This yields a daily log volume of:

$$500{,}000{,}000 \times 64 \text{ bytes} \approx 32 \text{ GB/day}.$$

Over one year the raw volume exceeds:

$$32 \text{ GB/day} \times 365 \approx 11.6 \text{ TB/year},$$

which clearly places telemetry in the "big data" category and motivates the use of a distributed file system and a SQL-on-files engine for OLAP workloads, instead of a traditional OLTP database.

4 Data Flow Diagram

To describe how data flows between the main processes and data stores, this section uses a Data Flow Diagram (DFD) in Yourdon–DeMarco notation. Circles represent processing modules, rectangles represent data stores, and arrows indicate the direction of data flow.

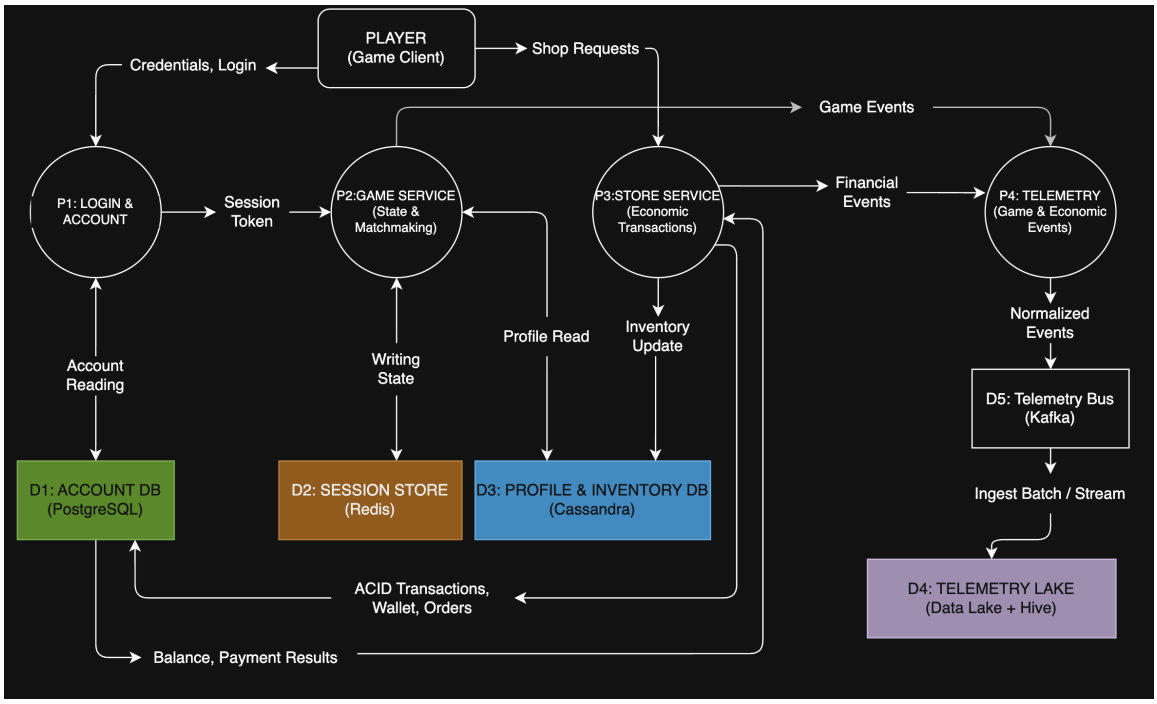

Figure 1: Data Flow Diagram for the Loca Lucha backend.

The DFD in Figure 1 models four main processes:

- **P1 – Login & Account**: authenticates the player and reads account data from the relational store.

- **P2 – Game Service (State & Matchmaking)**: manages gameplay sessions, matchmaking and PvP arenas, reading and writing character state.
- **P3 – Store Service (Economic Transactions)**: processes cash-shop purchases and premium currency operations.
- **P4 – Telemetry (Game & Economic Events)**: collects and normalizes gameplay and financial events before sending them to the analytics pipeline.

Each process interacts with one or more data stores:

- **D1 – Account DB (PostgreSQL)**: holds account records, wallet balances and transaction logs. P1 reads account data and P3 executes ACID transactions against this store.
- **D2 – Session Store (Redis)**: keeps hot session state, matchmaking queues and leaderboards entirely in memory. P2 performs very frequent reads and writes on this store.
- **D3 – Profile & Inventory DB (Cassandra)**: stores persistent character profiles and inventories. P2 loads profiles at login and persists durable state changes; P3 updates inventories after successful purchases.
- **D5 – Telemetry Bus (Kafka/Kinesis)**: temporary buffer for normalized events produced by P2 and P3.
- **D4 – Telemetry Lake (Data Lake + Hive)**: long-term storage for telemetry and log data, fed in batch/streaming mode from D5 and queried by analytics jobs.

From the player's point of view, the typical flows are:

- *Login*: the client sends credentials to P1, which validates them against D1 and passes a session token to P2.
- *Gameplay*: P2 loads the character profile from D3, maintains session state and leaderboards in D2, and emits telemetry events to P4.
- *Store purchase*: the client sends a shop request to P3; P3 executes an ACID transaction on D1 and, on success, updates the character inventory in D3.
- *Telemetry*: P2 and P3 send game and financial events to P4, which writes them to D5; from there they are ingested into D4 for offline analysis.

This DFD makes explicit that there is no single "central" database: different data stores are used in parallel, each serving the subset of data and operations for which it is best suited.

5 Storage Technologies and Rationale

This section explains which storage technology is used for each data store in Figure 1, and why alternative solutions would be a poor fit. The focus is on the match between data requirements and database properties, rather than on implementation details.

5.1 D1 – Account DB: PostgreSQL (Relational OLTP)

Account data and financial transactions require strong consistency, support for multi-statement transactions, integrity constraints and durable logging. A relational DBMS such as PostgreSQL, configured as an OLTP cluster, naturally provides:

- full ACID semantics for wallet updates and cash-shop operations;
- declarative constraints (primary/foreign keys, uniqueness, checks) to enforce referential integrity;
- well-understood backup and recovery procedures.

Under the CAP lens, this subsystem is effectively treated as *CP*: in the presence of network partitions, it is preferable to reject or delay a payment than to risk lost or duplicated money.

Bad ideas. A bad idea here would be to store real-money transactions directly in a generic key–value store or in Cassandra with eventual consistency: neither provides the same transactional guarantees or SQL-level integrity checks, and modelling complex financial workflows on top of them would add unnecessary complexity without any real benefit in terms of volume or throughput.

5.2 D2 – Session Store: Redis (In-Memory Key–Value)

Session tokens, matchmaking queues and PvP leaderboards are hot data with very high read-/write rates and strict latency requirements, but limited lifetime and small total volume. Redis, used as an in-memory key–value store, offers:

- sub-millisecond access times for reads and writes;
- specialised data structures (Sorted Sets) to implement global and per-segment leaderboards efficiently;
- simple expiry policies for ephemeral session keys.

In case of a crash, losing a small amount of transient matchmaking state is acceptable, because the definitive results of matches and rewards are persisted elsewhere (in PostgreSQL and Cassandra).

Bad ideas. A bad idea here would be to use only the relational DB for sessions and leaderboards. Even with proper indexing, storing every leaderboard update on disk would create heavy I/O pressure and locking overhead under peak load, translating into noticeable lag during PvP matches. Another bad idea would be to use Redis as the *only* source of truth for player state or money, since an in-memory cache is not designed to guarantee long-term durability.

5.3 D3 – Profile & Inventory DB: Apache Cassandra (Wide-Column)

Character profiles and inventories form a large, write-intensive, geographically distributed dataset. Apache Cassandra, as a wide-column NoSQL store, provides:

- a flexible schema, allowing denormalised "wide rows" per character;
- a masterless architecture with automatic sharding and replication across data centers;
- tunable consistency levels (e.g. LOCAL_QUORUM) to balance latency and consistency inside each region.

This design allows zone servers to read and write character state at high throughput while keeping the game available even if intercontinental links are temporarily degraded.

Bad ideas. A bad idea here would be to store all character state in a single relational schema and rely heavily on joins across many tables: with millions of players and frequent updates, join-heavy queries would quickly become a scalability bottleneck. Another bad idea would be to use a pure key–value store, which does not offer Cassandra's partitioning and clustering features for efficient range queries or compound keys based on player and character identifiers.

5.4 D4 – Telemetry Lake: Data Lake + Apache Hive

Telemetry and log data are append-only, high-volume and mostly consumed by batch analytics. Storing them in a Data Lake (e.g. HDFS or cloud object storage) and querying them with Apache Hive offers:

- cheap, horizontally scalable storage for tens of terabytes of data;
- a columnar on-disk layout and SQL-like language (HiveQL) for efficient OLAP queries over billions of rows;
- a clean separation between operational databases and analytical workloads.

Events are first written to the telemetry bus (Kafka/Kinesis), then ingested into large, compressed files in the lake, following best practices for game analytics pipelines.

Bad ideas. A bad idea here would be to store long-term telemetry inside the PostgreSQL cluster: full-table scans and heavy aggregations would compete with OLTP transactions for I/O and cache, increasing latency for gameplay-critical operations. Using Cassandra as a generic "sink" for all raw logs would also be suboptimal, as it lacks the SQL-on-files capabilities and ecosystem of tools available around Hive and data lakes for complex analytical processing.

6 Record Examples

This section provides a few concrete examples of how data items look like at storage level. The goal is not to describe a full schema, but to show the typical structure and format of the main records.

6.1 Character Profile in Cassandra

A persistent character profile, stored as a single wide row in Cassandra, can be represented in JSON as follows:

```
{
  "playerId": 4242,
  "characterId": 3,
  "shard": "EU-West-1",
  "level": 57,
  "class": "Mage",
  "stats": {
    "strength": 12,
    "intelligence": 145,
    "agility": 73
  },
  "inventory": [
    {"itemId": 10012, "name": "Arcane Staff", "qty": 1},
    {"itemId": 20007, "name": "Health Potion", "qty": 25}
  ],
  "quests": [
```

```
    {"questId": 501, "status": "COMPLETED"},
    {"questId": 742, "status": "IN_PROGRESS"}
  ],
  "position": {
    "world": "Azuria",
    "x": 123.4,
    "y": 87.6
  },
  "lastLoginTs": 1719298836
}
```

6.2 Telemetry Event in the Data Lake

An example of gameplay telemetry event sent to the Data Lake, in CSV format, is the following:

```
playerId,timestamp,eventType,details
4242,1719298836,MONSTER_KILL,"monsterId=501;zone=Azuria_Forest"
```

6.3 Session and Leaderboard Entries in Redis

Redis stores both session state and leaderboard ranks as key–value structures in memory. A typical session token can be represented as:

```
Key:    session:2f9c3a
Value:  userId=4242;shard=EU-West-1;expires=1719299000
```

Similarly, a PvP leaderboard entry is stored in a Sorted Set, where the score is the sorted-set member score and the player identifier is the member value:

```
ZADD pvp:leaderboard 1875 "player:4242"
```

These examples highlight that Redis data is small, ephemeral and optimised for fast in-memory access, rather than for long-term durability.

6.4 Account Record in PostgreSQL

For completeness, an account record in the relational store can be represented as:

```
{
  "userId": 9001,
  "email": "player9001@example.com",
  "passwordHash": "7c4a8d09ca3762af61e59520943dc26494f8941b",
  "walletBalance": 37.50,
  "registrationTs": 1683120452,
  "status": "ACTIVE"
}
```

These simple examples highlight that account and profile data have a structured schema, while telemetry events are append-only records with a lightweight, semi-structured payload, which further motivates the different storage choices adopted in the architecture.

7 Discussion and Related Work

The design of Loca Lucha closely follows patterns proposed in the literature for data management in MMORPGs. Diao and Schallehn classify game data into four sets — Account, Game, State and Log data — and show that a single relational DBMS cannot simultaneously meet the consistency, performance, availability and scalability requirements of all four [1]. They specifically recommend keeping Account data on an RDBMS while moving State and Log data to cloud-based storage systems such as Apache Cassandra, which offer flexible schemas, write-optimised storage and tunable consistency [1, 2].

The architecture presented in this homework is a direct instantiation of that approach: PostgreSQL is used for account and financial data, Cassandra for persistent character state, and a Data Lake with Hive for telemetry and log data. Redis is added as an in-memory layer for hot session and PvP state, a natural extension of the in-memory + disk-backed pattern described in those works.

Industrial practice points in the same direction. Public talks from Riot Games describe how League of Legends ingests large volumes of gameplay data into a Hadoop-based platform and uses Hive as a data warehouse to analyse champion balance and matchmaking across regions [3]. Cloud providers likewise recommend Kafka-style ingestion layers feeding data lakes for game telemetry. In this sense, the Loca Lucha backend can be seen as a compact polyglot-persistence design that aligns both with academic proposals and with architectures adopted in real-world online games.

8 Conclusions

This homework presented a polyglot persistence architecture for the backend of Loca Lucha, a hypothetical global-scale MMORPG. Starting from a classification of data into Account, Game, State and Log classes, the analysis quantified the expected volumes and access patterns and showed that a single monolithic DBMS would either over-provision for some workloads or underdeliver for others.

By assigning each class of data to a specialised storage technology the design balances consistency, latency, availability and scalability in a way that matches real-world MMO requirements.

The main trade-off is the increased operational complexity of running and monitoring multiple data stores. However, this complexity is justified by the ability to avoid the bottlenecks of a "one size fits all" database and to align each subset of data with the persistence model that best serves it. Future extensions could add further specialised stores (for example, a graph database for social relationships or a feature store for recommendation models) following the same guiding principle: choose the right tool for each data class rather than forcing all of them into a single technology.

References

[1] Z. Diao and E. Schallehn. Towards Cloud Data Management for MMORPGs. In *Proc. of the 3rd International Conference on Cloud Computing and Services Science (CLOSER)*, pages 303–308, 2013.

[2] Z. Diao, P. Zhao, E. Schallehn, and S. Mohammad. Achieving Consistent Storage for Scalable MMORPG Environments. In *Proc. of the 19th International Database Engineering & Applications Symposium (IDEAS)*, 2015.

[3] Riot Games. Big Data at Riot Games. Hadoop Summit 2012 talk and slides, available online, 2012.